

AVALIAÇÃO OBJETIVA - RESUMO - T DESS 2025/1 V1

Este resumo consolida os principais tópicos abordados nas aulas, organizados desde os fundamentos teóricos até as práticas avançadas de teste e qualidade de software.

1. Fundamentos e Mentalidade de Qualidade

A qualidade de software evoluiu do simples ato de "ver se funciona" (debugging) para a Engenharia de Qualidade, onde o foco é a prevenção. O teste é definido como o processo de executar um programa com a intenção expressa de encontrar erros.

- **A Tríade Crítica:** É essencial distinguir entre **Erro** (ação humana equivocada), **Defeito/Bug** (manifestação física do erro no código) e **Falha** (desvio do comportamento esperado durante a execução).
- **Shift-Left e Custo do Defeito:** A cultura *Shift-Left* propõe mover os testes para o início do projeto. Segundo a "Regra de 10", o custo de correção aumenta exponencialmente conforme a fase em que o defeito é descoberto, podendo ser 100 vezes mais caro em produção do que na fase de requisitos.

2. Papéis, Estrutura e Níveis de Teste

O mercado diferencia o **Testador** (focado no produto final) do **QA** (focado na melhoria do processo) e do **Engenheiro de Qualidade** (focado em automação).

- **Pirâmide de Testes:** Uma estratégia eficiente de automação deve ter uma base larga de **Testes Unitários** (rápidos e baratos), um meio com **Testes de Integração** e um topo estreito com **Testes de UI/Sistema** (lentos e caros).
- **Níveis de Teste:** Os testes são divididos em Unitário (menor parte funcional), Integração (interação entre módulos), Sistema (aplicação completa de ponta a ponta) e Aceite (validado pelo usuário final).

3. Planejamento e Análise de Requisitos

O planejamento evita o caos e gerencia incertezas.

- **Requisitos:** Dividem-se em **Funcionais (RF)** (o que o sistema faz) e **Não Funcionais (RNF)** (como o sistema se comporta em termos de performance, segurança e usabilidade).
- **BDD (Behavior Driven Development):** Utiliza a sintaxe **Gherkin** (Dado, Quando, Então) para criar uma linguagem comum entre negócio e técnicos, tornando os requisitos testáveis e executáveis.
- **Plano de Testes:** Documento mestre que define o escopo (positivo e negativo), critérios de parada, ferramentas e riscos.

4. Métodos, Técnicas e Execução

- **Estratégias de Visibilidade:** **Caixa-Branca** (foca na lógica interna do código), **Caixa-Preta** (foca em entradas e saídas via interface) e **Caixa-Cinza** (abordagem híbrida).
- **Técnicas de Design:** **Particionamento por Classes de Equivalência** (dividir entradas em grupos tratados da mesma forma) e **Análise do Valor Limite (BVA)** (testar as fronteiras onde erros são mais comuns).
- **Documentação de Defeitos:** Um **Bug Report** rico deve ser neutro e conter título, passos para reprodução (essencial para o desenvolvedor), resultados esperado e obtido, e evidências.
- **Rastreabilidade:** A Matriz de Rastreabilidade (RTM) conecta requisitos a casos de teste e defeitos, garantindo a cobertura total do projeto.

5. Práticas Modernas: TDD e Automação

- **TDD (Test-Driven Development):** Segue o ciclo **Red** (escrever teste que falha), **Green** (escrever código mínimo para passar) e **Refactor** (melhorar o código).
- **Padrão AAA:** Organiza a anatomia de um teste em **Arrange** (preparar cenário), **Act** (executar ação) e **Assert** (verificar resultado).

6. Testes Não Funcionais (UX, Performance e Segurança)

- **Performance:** Inclui testes de carga (dentro do limite esperado) e estresse (além do limite para achar o ponto de ruptura ou gargalo). Métricas comuns são Latência e Throughput.
- **UX e Acessibilidade:** Baseia-se nas **10 Heurísticas de Nielsen** para usabilidade e nas diretrizes **WCAG** para acessibilidade, seguindo os princípios **POUR** (Perceptível, Operável, Compreensível e Robusto).
- **Segurança:** Foca no mapeamento de vulnerabilidades como **SQL Injection** e **XSS**, utilizando o ranking **OWASP Top 10** como guia principal. As abordagens incluem **SAST** (análise estática) e **DAST** (análise dinâmica).